



Covid Data Analysis

Experiment - 19 & 20

Gunajn Goyal

25070123048

E&TC - A3

```
In [ ]: import pandas as pd
import numpy as np
```

```
In [ ]: data = pd.read_csv("/content/covid_19_data.csv")
data.head()
```

```
Out[ ]:
```

	SNo	ObservationDate	Province/ State	Country/ Region	Last Update	Confirmed	Deaths	Rec
0	1	01/22/2020	Anhui	Mainland China	1/22/ 2020 17:00	1.0	0.0	
1	2	01/22/2020	Beijing	Mainland China	1/22/ 2020 17:00	14.0	0.0	
2	3	01/22/2020	Chongqing	Mainland China	1/22/ 2020 17:00	6.0	0.0	
3	4	01/22/2020	Fujian	Mainland China	1/22/ 2020 17:00	1.0	0.0	
4	5	01/22/2020	Gansu	Mainland China	1/22/ 2020 17:00	0.0	0.0	

```
In [ ]: data.tail()
```

Out[]:

	SNo	ObservationDate	Province/ State	Country/ Region	Last Update	Confirmed
306424	306425	05/29/2021	Zaporizhia Oblast	Ukraine	2021-05-30 04:20:55	102641.0
306425	306426	05/29/2021	Zeeland	Netherlands	2021-05-30 04:20:55	29147.0
306426	306427	05/29/2021	Zhejiang	Mainland China	2021-05-30 04:20:55	1364.0
306427	306428	05/29/2021	Zhytomyr Oblast	Ukraine	2021-05-30 04:20:55	87550.0
306428	306429	05/29/2021	Zuid- Holland	Netherlands	2021-05-30 04:20:55	391559.0

```
In [ ]: data=data.drop(['SNo','Last Update'],axis=1)
```

```
In [ ]: data.head()
```

Out[]:

	ObservationDate	Province/ State	Country/ Region	Confirmed	Deaths	Recovered
0	01/22/2020	Anhui	Mainland China	1.0	0.0	0.0
1	01/22/2020	Beijing	Mainland China	14.0	0.0	0.0
2	01/22/2020	Chongqing	Mainland China	6.0	0.0	0.0
3	01/22/2020	Fujian	Mainland China	1.0	0.0	0.0
4	01/22/2020	Gansu	Mainland China	0.0	0.0	0.0

```
In [ ]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 306429 entries, 0 to 306428
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   ObservationDate  306429 non-null object  
1   Province/State  228326 non-null object  
2   Country/Region  306429 non-null object  
3   Confirmed       306429 non-null float64 
4   Deaths         306429 non-null float64 
5   Recovered       306429 non-null float64 
dtypes: float64(3), object(3)
memory usage: 14.0+ MB
```

```
In [ ]: data['ObservationDate'] = data['ObservationDate'].astype('datetime64[ns]')
data['Confirmed'] = data['Confirmed'].astype('int64')
data['Deaths'] = data['Deaths'].astype('int64')
data['Recovered'] = data['Recovered'].astype('int64')
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 306429 entries, 0 to 306428
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ObservationDate        306429 non-null  datetime64[ns]
1   Province/State        228326 non-null  object
2   Country/Region        306429 non-null  object
3   Confirmed             306429 non-null  int64
4   Deaths               306429 non-null  int64
5   Recovered             306429 non-null  int64
dtypes: datetime64[ns](1), int64(3), object(2)
memory usage: 14.0+ MB
```

```
In [ ]: data.head()
```

```
Out[ ]:
```

	ObservationDate	Province/ State	Country/ Region	Confirmed	Deaths	Recovered
0	2020-01-22	Anhui	Mainland China	1	0	0
1	2020-01-22	Beijing	Mainland China	14	0	0
2	2020-01-22	Chongqing	Mainland China	6	0	0
3	2020-01-22	Fujian	Mainland China	1	0	0
4	2020-01-22	Gansu	Mainland China	0	0	0

```
In [ ]: # Active Cases:
data['Active'] = data['Confirmed'] - data['Recovered'] - data['Deaths']
data.head(50)

data.iloc[50:100]
```

Out[]:

	ObservationDate	Province/ State	Country/ Region	Confirmed	Deaths	Recovered	Ac
50	2020-01-23	Heilongjiang	Mainland China	2	0	0	
51	2020-01-23	Henan	Mainland China	5	0	0	
52	2020-01-23	Hong Kong	Hong Kong	2	0	0	
53	2020-01-23	Hubei	Mainland China	444	17	28	
54	2020-01-23	Hunan	Mainland China	9	0	0	
55	2020-01-23	Inner Mongolia	Mainland China	0	0	0	
56	2020-01-23	Jiangsu	Mainland China	5	0	0	
57	2020-01-23	Jiangxi	Mainland China	7	0	0	
58	2020-01-23	Jilin	Mainland China	1	0	0	
59	2020-01-23	Liaoning	Mainland China	3	0	0	
60	2020-01-23	Macau	Macau	2	0	0	
61	2020-01-23	Ningxia	Mainland China	1	0	0	
62	2020-01-23	Qinghai	Mainland China	0	0	0	
63	2020-01-23	Shaanxi	Mainland China	3	0	0	
64	2020-01-23	Shandong	Mainland China	6	0	0	
65	2020-01-23	Shanghai	Mainland China	16	0	0	
66	2020-01-23	Shanxi	Mainland China	1	0	0	
67	2020-01-23	Sichuan	Mainland China	8	0	0	
68	2020-01-23	Taiwan	Taiwan	1	0	0	
69	2020-01-23	Tianjin	Mainland China	4	0	0	

	ObservationDate	Province/ State	Country/ Region	Confirmed	Deaths	Recovered	Ac
70	2020-01-23	Tibet	Mainland China	0	0	0	
71	2020-01-23	Washington	US	1	0	0	
72	2020-01-23	Xinjiang	Mainland China	2	0	0	
73	2020-01-23	Yunnan	Mainland China	2	0	0	
74	2020-01-23	Zhejiang	Mainland China	27	0	0	
75	2020-01-23	NaN	Japan	1	0	0	
76	2020-01-23	NaN	Thailand	4	0	2	
77	2020-01-23	NaN	South Korea	1	0	0	
78	2020-01-23	NaN	Singapore	1	0	0	
79	2020-01-23	NaN	Philippines	0	0	0	
80	2020-01-23	NaN	Malaysia	0	0	0	
81	2020-01-23	NaN	Vietnam	2	0	0	
82	2020-01-23	NaN	Australia	0	0	0	
83	2020-01-23	NaN	Mexico	0	0	0	
84	2020-01-23	NaN	Brazil	0	0	0	
85	2020-01-23	NaN	Colombia	0	0	0	
86	2020-01-23	Unknown	China	0	0	0	
87	2020-01-23	NaN	Kiribati	0	0	0	
88	2020-01-24	Hubei	Mainland China	549	24	31	
89	2020-01-24	Guangdong	Mainland China	53	0	2	
90	2020-01-24	Zhejiang	Mainland China	43	0	1	
91	2020-01-24	Beijing	Mainland China	36	0	1	
92	2020-01-24	Chongqing	Mainland China	27	0	0	
93	2020-01-24	Hunan	Mainland China	24	0	0	

	ObservationDate	Province/ State	Country/ Region	Confirmed	Deaths	Recovered	Ac
94	2020-01-24	Guangxi	Mainland China	23	0	0	
95	2020-01-24	Shanghai	Mainland China	20	0	1	
96	2020-01-24	Jiangxi	Mainland China	18	0	0	
97	2020-01-24	Sichuan	Mainland China	15	0	0	
98	2020-01-24	Shandong	Mainland China	15	0	0	
99	2020-01-24	Anhui	Mainland China	15	0	0	

```
In [ ]: # Last/Current Date:
data['ObservationDate'].max()
```

```
Out[ ]: Timestamp('2021-05-29 00:00:00')
```

```
In [ ]: # Fetch the latest date data
latest_data = data[data['ObservationDate'] == data['ObservationDate'].max()]
latest_data.head()
```

```
Out[ ]:
```

	ObservationDate	Province/ State	Country/ Region	Confirmed	Deaths	Recovered
305664	2021-05-29	NaN	Afghanistan	70111	2899	57281
305665	2021-05-29	NaN	Albania	132297	2449	129215
305666	2021-05-29	NaN	Algeria	128456	3460	89419
305667	2021-05-29	NaN	Andorra	13693	127	13416
305668	2021-05-29	NaN	Angola	34180	757	27646

```
In [ ]: latest_data.shape
```

```
Out[ ]: (765, 7)
```

```
In [ ]: latest_data['Country/Region'].value_counts()
```

Out[]: **count**

Country/Region	
Russia	83
US	58
Japan	49
India	37
Colombia	34
...	...
Venezuela	1
Vietnam	1
West Bank and Gaza	1
Hong Kong	1
Macau	1

195 rows × 1 columns

dtype: int64

```
In [ ]: latest_data['Country/Region'].nunique()
```

Out[]: 195

```
In [ ]: latest_data['Country/Region'].unique()
```

```
Out[ ]: array(['Afghanistan', 'Albania', 'Algeria', 'Andorra', 'Angola',
               'Antigua and Barbuda', 'Argentina', 'Armenia', 'Austria',
               'Azerbaijan', 'Bahamas', 'Bahrain', 'Bangladesh', 'Barbados',
               'Belarus', 'Belize', 'Benin', 'Bhutan', 'Bolivia',
               'Bosnia and Herzegovina', 'Botswana', 'Brunei', 'Bulgaria',
               'Burkina Faso', 'Burma', 'Burundi', 'Cabo Verde', 'Cambodia',
               'Cameroon', 'Central African Republic', 'Chad', 'Comoros',
               'Congo (Brazzaville)', 'Congo (Kinshasa)', 'Costa Rica', 'Croatia',
               'Cuba', 'Cyprus', 'Czech Republic', 'Denmark', 'Diamond Princess',
               'Djibouti', 'Dominica', 'Dominican Republic', 'Ecuador', 'Egypt',
               'El Salvador', 'Equatorial Guinea', 'Eritrea', 'Estonia',
               'Eswatini', 'Ethiopia', 'Fiji', 'Finland', 'France', 'Gabon',
               'Gambia', 'Georgia', 'Ghana', 'Greece', 'Grenada', 'Guatemala',
               'Guinea', 'Guinea-Bissau', 'Guyana', 'Haiti', 'Holy See',
               'Honduras', 'Hungary', 'Iceland', 'Indonesia', 'Iran', 'Iraq',
               'Ireland', 'Israel', 'Ivory Coast', 'Jamaica', 'Jordan',
               'Kazakhstan', 'Kenya', 'Kiribati', 'Kosovo', 'Kuwait',
               'Kyrgyzstan', 'Laos', 'Latvia', 'Lebanon', 'Lesotho', 'Liberia',
               'Libya', 'Liechtenstein', 'Lithuania', 'Luxembourg', 'MS Zaandam',
               'Madagascar', 'Malawi', 'Malaysia', 'Maldives', 'Mali', 'Malta',
               'Marshall Islands', 'Mauritania', 'Mauritius', 'Micronesia',
               'Moldova', 'Monaco', 'Mongolia', 'Montenegro', 'Morocco',
               'Mozambique', 'Namibia', 'Nepal', 'New Zealand', 'Nicaragua',
               'Niger', 'Nigeria', 'North Macedonia', 'Norway', 'Oman', 'Panama',
               'Papua New Guinea', 'Paraguay', 'Philippines', 'Poland',
               'Portugal', 'Qatar', 'Romania', 'Rwanda', 'Saint Kitts and Nevis',
               'Saint Lucia', 'Saint Vincent and the Grenadines', 'Samoa',
               'San Marino', 'Sao Tome and Principe', 'Saudi Arabia', 'Senegal',
               'Serbia', 'Seychelles', 'Sierra Leone', 'Singapore', 'Slovakia',
               'Slovenia', 'Solomon Islands', 'Somalia', 'South Africa',
               'South Korea', 'South Sudan', 'Sri Lanka', 'Sudan', 'Suriname',
               'Switzerland', 'Syria', 'Taiwan', 'Tajikistan', 'Tanzania',
               'Thailand', 'Timor-Leste', 'Togo', 'Trinidad and Tobago',
               'Tunisia', 'Turkey', 'Uganda', 'United Arab Emirates', 'Uruguay',
               'Uzbekistan', 'Vanuatu', 'Venezuela', 'Vietnam',
               'West Bank and Gaza', 'Yemen', 'Zambia', 'Zimbabwe', 'Italy',
               'Brazil', 'Russia', 'Mexico', 'Japan', 'US', 'Canada', 'Colombia',
               'Peru', 'Spain', 'India', 'UK', 'Mainland China', 'Chile',
               'Belgium', 'Netherlands', 'Australia', 'Pakistan', 'Germany',
               'Sweden', 'Ukraine', 'Hong Kong', 'Macau'], dtype=object)
```

```
In [ ]: # Counts of cases for each country:
```

```
countries = latest_data.groupby("Country/Region")[["Confirmed", "Deaths", "Active"]]
countries = countries.reset_index()
```

```
In [ ]: countries
```

Out[]:

	Country/Region	Confirmed	Deaths	Active
0	Afghanistan	70111	2899	9931
1	Albania	132297	2449	633
2	Algeria	128456	3460	35577
3	Andorra	13693	127	150
4	Angola	34180	757	5777
...	...	...	...	...
190	Vietnam	6908	47	3965
191	West Bank and Gaza	307838	3492	3822
192	Yemen	6731	1319	2013
193	Zambia	94751	1276	1881
194	Zimbabwe	38933	1594	761

195 rows × 4 columns

In []: *#Cases in China*

```
countries[countries['Country/Region']=="Mainland China"]
```

Out[]:

	Country/Region	Confirmed	Deaths	Active
107	Mainland China	91072	4636	319

In []: *#Cases in India*

```
countries[countries['Country/Region']=="India"]
```

Out[]:

	Country/Region	Confirmed	Deaths	Active
78	India	27894800	325972	2114508

In []: *#Cases in Spain*

```
countries[countries['Country/Region']=="Spain"]
```

Out[]:

	Country/Region	Confirmed	Deaths	Active
165	Spain	3668658	79905	3438377

In []: *#Cases in Germany*

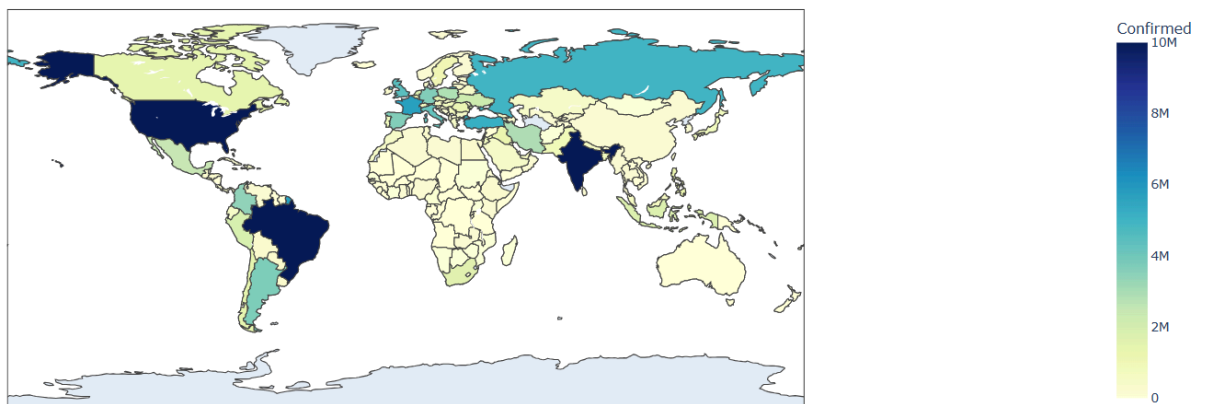
```
countries[countries['Country/Region']=="Germany"]
```

```
Out[ ]:      Country/Region  Confirmed  Deaths  Active
64      Germany      3684672    88413   116559
```

```
In [ ]: # Plot on World Map
```

```
import plotly.express as px
import plotly.graph_objects as go
```

```
In [ ]: world_map = px.choropleth(countries, locations = "Country/Region", locationmode='ne',
                                color="Confirmed", color_continuous_scale="YlGnBu", range=[0, 10000000])
world_map
```



```
In [ ]: #Top 20 Country Region
```

```
top = latest_data.groupby("Country/Region")[['Confirmed', 'Recovered']].sum().reset_index()
top = top.sort_values(['Confirmed'], ascending=False)
top.head()
```

Out[]:

	Country/Region	Confirmed	Recovered
--	----------------	-----------	-----------

182	US	33251939	0
78	India	27894800	25454320
23	Brazil	16471600	14496224
60	France	5719877	390878
180	Turkey	5235978	5094279

In []: `top_20 = top.head(20)`
`top_20`

Out[]:

	Country/Region	Confirmed	Recovered
--	----------------	-----------	-----------

182	US	33251939	0
78	India	27894800	25454320
23	Brazil	16471600	14496224
60	France	5719877	390878
180	Turkey	5235978	5094279
144	Russia	4995613	4616422
181	UK	4496823	15481
84	Italy	4213055	3845087
6	Argentina	3732263	3288467
64	Germany	3684672	3479700
165	Spain	3668658	150376
36	Colombia	3363061	3141549
80	Iran	2893218	2425033
140	Poland	2871371	2636675
116	Mexico	2411503	1924865
184	Ukraine	2257904	2084477
138	Peru	1947555	1897522
79	Indonesia	1809926	1659974
126	Netherlands	1671967	26810
44	Czech Republic	1660935	1617498

```
In [ ]: spain = data[data['Country/Region']=='Spain'].copy()
        spain
```

```
Out[ ]:
```

	ObservationDate	Province/ State	Country/ Region	Confirmed	Deaths	Recovered
571	2020-02-01	NaN	Spain	1	0	0
640	2020-02-02	NaN	Spain	1	0	0
711	2020-02-03	NaN	Spain	1	0	0
783	2020-02-04	NaN	Spain	1	0	0
855	2020-02-05	NaN	Spain	1	0	0
...	...	...	...	...	...	...
306143	2021-05-29	Melilla	Spain	9098	97	125
306165	2021-05-29	Murcia	Spain	112926	1600	2180
306174	2021-05-29	Navarra	Spain	62232	1171	3905
306229	2021-05-29	Pais Vasco	Spain	197812	4429	16160
306372	2021-05-29	Unknown	Spain	0	0	0

7718 rows × 7 columns

```
In [ ]: # Find total number of States
        spain['Province/State'].nunique()
```

```
Out[ ]: 20
```

```
In [ ]: spain['Province/State'].unique()
```

```
Out[ ]: array([nan, 'Andalusia', 'Aragon', 'Asturias', 'Balears',
               'C. Valenciana', 'Canarias', 'Cantabria', 'Castilla - La Mancha',
               'Castilla y Leon', 'Catalonia', 'Ceuta', 'Extremadura', 'Galicia',
               'La Rioja', 'Madrid', 'Melilla', 'Murcia', 'Navarra', 'Pais Vasco',
               'Unknown'], dtype=object)
```

```
In [ ]: # Calculate the mode of 'Province/State'
        mode_province_state = spain['Province/State'].mode()[0]
        print(f"The mode of 'Province/State' is: {mode_province_state}")
```

The mode of 'Province/State' is: Andalusia

```
In [ ]: spain['Province/State'].unique()
```

```
Out[ ]: array([nan, 'Andalusia', 'Aragon', 'Asturias', 'Balears',  
              'C. Valenciana', 'Canarias', 'Cantabria', 'Castilla - La Mancha',  
              'Castilla y Leon', 'Catalonia', 'Ceuta', 'Extremadura', 'Galicia',  
              'La Rioja', 'Madrid', 'Melilla', 'Murcia', 'Navarra', 'Pais Vasco',  
              'Unknown'], dtype=object)
```

```
In [ ]: #Spain Latest Data:
```

```
spain_latest_data = spain[spain['ObservationDate'] == spain['ObservationDate']  
spain_latest_data
```

```
Out[ ]:
```

	ObservationDate	Province/ State	Country/ Region	Confirmed	Deaths	Recovered
305854	2021-05-29	Andalusia	Spain	581880	9880	10671
305864	2021-05-29	Aragon	Spain	123905	3504	3772
305876	2021-05-29	Asturias	Spain	52213	1965	1063
305887	2021-05-29	Balears	Spain	60364	839	1533
305909	2021-05-29	C. Valenciana	Spain	394360	7389	9970
305917	2021-05-29	Canarias	Spain	56226	769	1537
305918	2021-05-29	Cantabria	Spain	30178	564	2287
305922	2021-05-29	Castilla - La Mancha	Spain	192347	5912	6392
305923	2021-05-29	Castilla y Leon	Spain	230600	6882	8716
305924	2021-05-29	Catalonia	Spain	611008	14531	26203
305929	2021-05-29	Ceuta	Spain	5829	114	163
305974	2021-05-29	Extremadura	Spain	75693	1801	2652
305988	2021-05-29	Galicia	Spain	126622	2402	9204
306099	2021-05-29	La Rioja	Spain	30749	771	3107
306123	2021-05-29	Madrid	Spain	714616	15285	40736
306143	2021-05-29	Melilla	Spain	9098	97	125
306165	2021-05-29	Murcia	Spain	112926	1600	2180
306174	2021-05-29	Navarra	Spain	62232	1171	3905
306229	2021-05-29	Pais Vasco	Spain	197812	4429	16160
306372	2021-05-29	Unknown	Spain	0	0	0

```
In [ ]: #Top 20 Province/Sate

top_state = spain_latest_data.groupby("Province/State")[['Confirmed', 'Recovered']]
top_state = top_state.sort_values(['Confirmed'], ascending=False)
top_state.head(20)
```

```
Out[ ]:
```

	Province/State	Confirmed	Recovered
14	Madrid	714616	40736
9	Catalonia	611008	26203
0	Andalusia	581880	10671
4	C. Valenciana	394360	9970
8	Castilla y Leon	230600	8716
18	Pais Vasco	197812	16160
7	Castilla - La Mancha	192347	6392
12	Galicia	126622	9204
1	Aragon	123905	3772
16	Murcia	112926	2180
11	Extremadura	75693	2652
17	Navarra	62232	3905
3	Baleares	60364	1533
5	Canarias	56226	1537
2	Asturias	52213	1063
13	La Rioja	30749	3107
6	Cantabria	30178	2287
15	Melilla	9098	125
10	Ceuta	5829	163
19	Unknown	0	0

```
In [ ]: top_state['Confirmed'].max()
```

```
Out[ ]: 714616
```

```
In [ ]: top_state[top_state['Confirmed']==top_state['Confirmed'].max()]['Province/State']
```

```
Out[ ]: 'Madrid'
```

```
In [ ]:
```

```
In [ ]: spain_total_cases = countries[countries['Country/Region'] == 'Spain']

fig = px.choropleth(spain_total_cases,
                    locations="Country/Region",
                    locationmode="country names",
                    color="Confirmed",
                    color_continuous_scale="Magma",
                    title="COVID-19 Confirmed Cases in Spain")

fig.show()
```

